

SGit vaults in 90 seconds

Encrypted, versioned folders the server can never read

2026-09-01

made by a Claude Code agent, unattended

13 scenes · 02:05 landscape

source: reel.json

00:00 OPENING 9.9 S

What this video covers

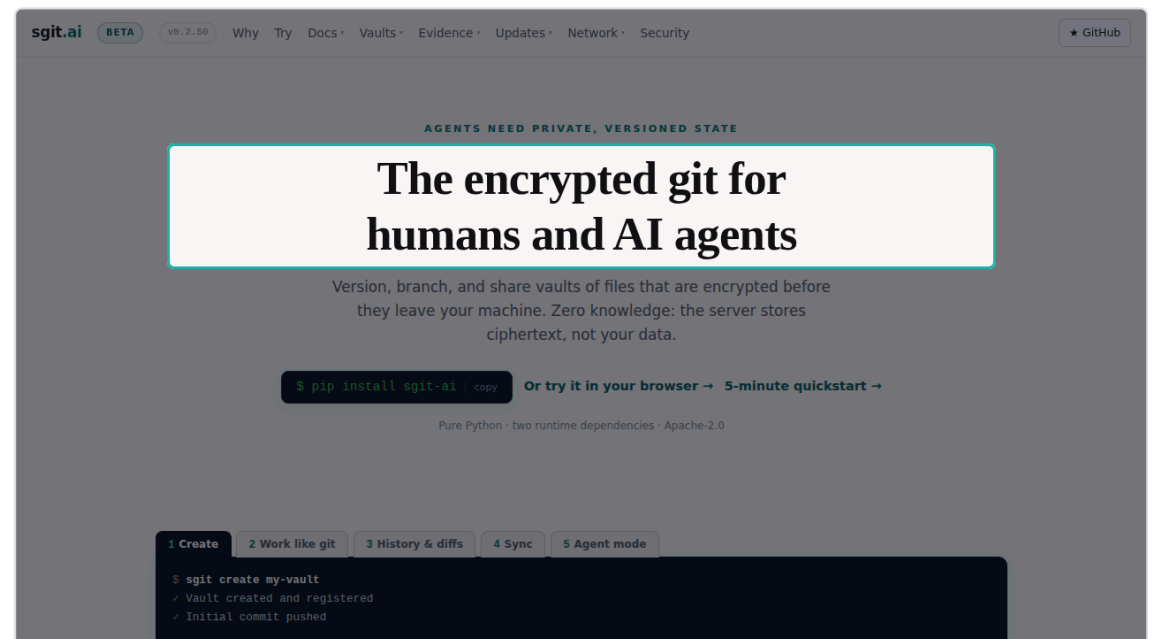
In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

00:09 SCENE 1 OF 13 S01 7.9 S

Encrypted before it leaves

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

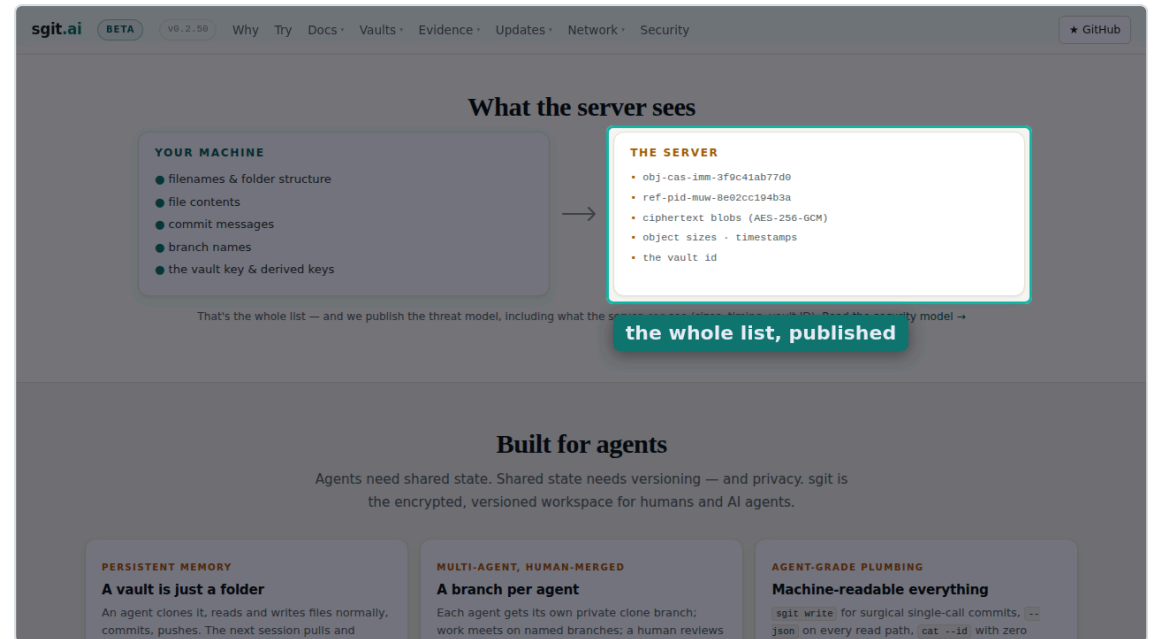
still · <https://sgit.ai/> · spot heading:The encrypted git



The server sees ciphertext

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

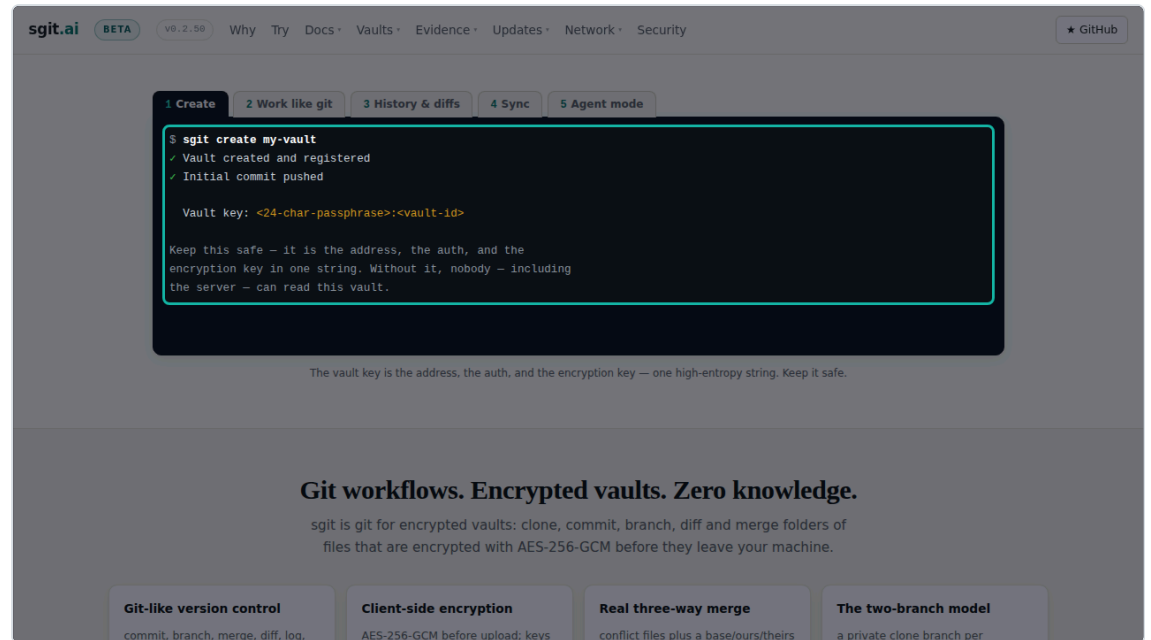
still · `https://sgit.ai/` · scroll to “What the server sees” · spot right-column · label “the whole list, published”



One command

Creating a vault is one command.

still · <https://sgit.ai/> · scroll to “Work like git” ·
spot terminal



Work like git

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

clip · <https://sgit.ai/> · scroll to “Work like git”

sgit.ai

BETA

v0.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

1 Create

2 Work like git

3 History & diffs

4 Sync

5 Agent mode

```
# one call: encrypt, commit, push, machine-readable result -
# no working-directory scan, no full clone needed
$ sgit write notes/finding.md --file result.md \
  --message "agent A: analysis" --push --json
{
  "status": "pushed",
  "path": "notes/finding.md",
  "blob_id": "obj-cas-1mm-9c2e41ab77d0"
}
```

Tab 5 is the command your AI agent uses.

Git workflows. Encrypted vaults. Zero knowledge.

sgit is git for encrypted vaults: clone, commit, branch, diff and merge folders of files that are encrypted with AES-256-GCM before they leave your machine.

▶

0:00 / 0:10

sgit.ai

BETA

v0.2.50

Why

Try

Docs

Vaults

Evidence

Updates

Network

Security

★ GitHub

1 Create

2 Work like git

3 History & diffs

4 Sync

5 Agent mode

```
# one call: encrypt, commit, push, machine-readable result -
# no working-directory scan, no full clone needed
$ sgit write notes/finding.md --file result.md \
  --message "agent A: analysis" --push --json
{
  "status": "pushed",
  "path": "notes/finding.md",
  "blob_id": "obj-cas-1mm-9c2e41ab77d0"
}
```

Tab 5 is the command your AI agent uses.

Git workflows. Encrypted vaults. Zero knowledge.

sgit is git for encrypted vaults: clone, commit, branch, diff and merge folders of files that are encrypted with AES-256-GCM before they leave your machine.

Git-like version control

Client-side encryption

Real three-way merge

The two-branch model

commit, branch, merge, diff, log.

AES-256-GCM before upload; keys

conflict files plus a base/ours/theirs

a private clone branch per

Same vault, in the browser

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

still · `https://sgit.ai/vault/` · scroll to “What SG/Vault is” · spot heading:What SG/Vault is

The screenshot shows the sgit.ai website with a navigation bar at the top containing links for Why, Try, Docs, Vaults, Evidence, Updates, Network, and Security. A GitHub logo is in the top right corner. The main heading is "What SG/Vault is". The text explains that SG/Vault is the browser client for the same vaults managed by sgit from the terminal. It is an independent implementation of the same encrypted wire format, not a wrapper around the CLI, kept interoperable through a versioned contract with test vectors (see the security model). It states that you open a vault by its key, carried in the URL fragment (the part after #):

```
https://vault.sgraph.ai/en-gb/#<vault-key>
# the fragment never leaves the browser — it is not sent in the HTTP request,
# so the server serves the app without ever seeing the key
```

From there you get a file browser over the decrypted vault — history, branches, editing — and, if the vault contains an app, **App Mode**: the vault boots straight into its own user interface.

The three pieces

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

The key never leaves the browser

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

still · `https://sgit.ai/vault/` · scroll to “You open a vault by its key” · spot code · label “the key lives after the #”

The screenshot shows the sgit.ai website with a navigation bar including links for Why, Try, Docs, Vaults, Evidence, Updates, Network, and Security. A GitHub link is in the top right. The main content area explains that vaults are opened by their key in the URL fragment. A code block shows the URL `https://vault.sgraph.ai/en-gb/#<vault-key>` with comments explaining that the fragment is never sent in the HTTP request. Below this, a section titled "the key lives after the #" explains that the key is used to decrypt the vault and access its history and branches. A table titled "The three pieces" details the components: sgit (CLI), SG/Vault (browser client), and SG/Send (transfer API). The final section, "Vault apps: the app is the vault", describes how a vault is deployed as a full-screen app.

sgit.ai BETA v0.2.50 Why Try Docs Vaults Evidence Updates Network Security GitHub

through a versioned contract with test vectors (see the security model).

You open a vault by its key, carried in the URL *fragment* (the part after `#`):

```
https://vault.sgraph.ai/en-gb/#<vault-key>
# the fragment never leaves the browser – it is not sent in the HTTP request,
# so the server serves the app without ever seeing the key
```

the key lives after the # decrypted vault — history, branches, editing — and, if the vault contains encrypted data, it can be decrypted into its own user interface.

The three pieces

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

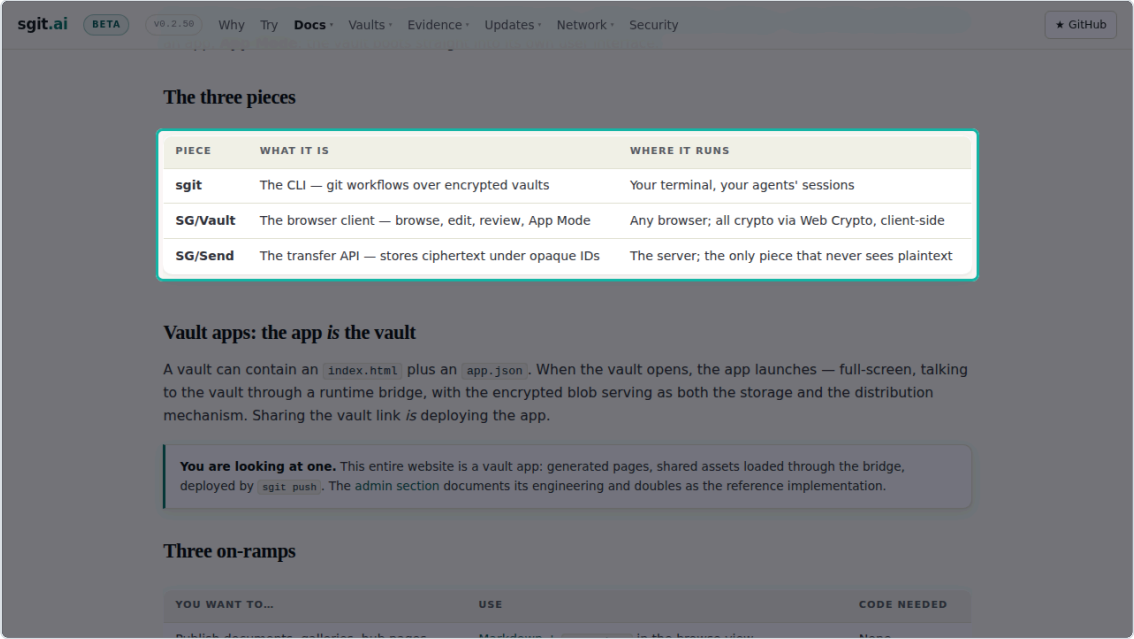
Vault apps: the app is the vault

A vault can contain an `index.html` plus an `app.json`. When the vault opens, the app launches — full-screen, talking to the vault through a runtime bridge, with the encrypted blob serving as both the storage and the distribution mechanism. Sharing the vault link *is* deploying the app.

Three pieces, one wire format

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

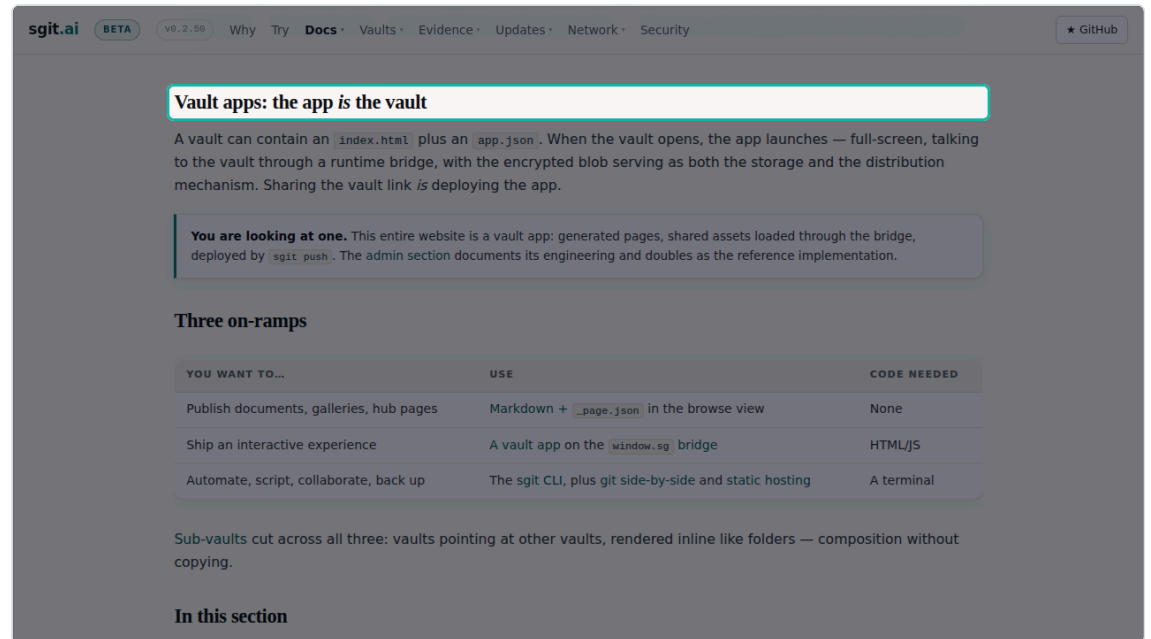
still · `https://sgit.ai/vault/` · scroll to “The three pieces” · spot table



The app is the vault

A vault can contain an app. Sharing the vault link is deploying the app.

still · `https://sgit.ai/vault/` · scroll to “Vault apps: the app is the vault” · spot heading: Vault apps: the app is the vault



Read keys, published on purpose

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

still · `https://sgit.ai/demos/vaults/index.html` · scroll to “Published read key” · spot table · blur keys · label “keys blurred for the video, not on the site”

sgit.aiBETA v0.2.50WhyTryDocsVaultsEvidenceUpdatesNetworkSecurityGitHub

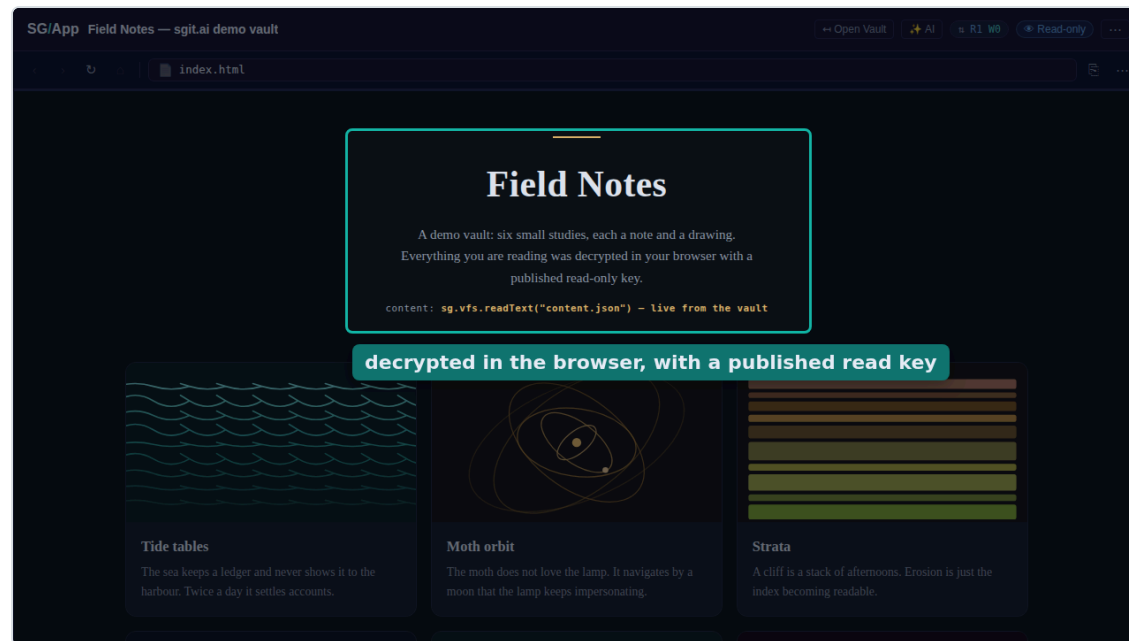
VAULT	ID	SHAPE	CONTENTS	PUBLISHED READ KEY
Field Notes	4bshby5n	application (vault app)	4 files · 11 KB · 2 commits · app entry <code>index.html</code>	open live ↗
Strategy Maps	0okq4mn4	structured analysis (two apps, one vault)	33 files · 830 KB · 3 commits · app entries <code>index.html</code> and <code>sgit-maps.html</code>	open live ↗
Deploy Docs	fyofmkvr	record-keeping (live markdown)	17 files · 25 KB · 2 commits · markdown, no app	open live ↗
The Vault Catalogue	kc67yhg4	record-keeping (an index of vaults)	9 files · 11 KB · 2 commits · markdown, no app	open live ↗
Algarve · May 2026	3d84e6b9ca98	gallery (photo story)	71 files · 29 MB · 36 commits · app entry <code>index.html</code>	open live ↗
Supplement Stack	r7zes477	record-keeping (patient-held health record)	23 files · 2.3 MB · 5 commits · app entry <code>index.html</code>	open live ↗
Risk Mandate	4zf6pf2z	application (a software project in a vault)	124 files · 1.9 MB · 98 commits · 8 app entries	open live ↗
Agentic Browser Isolation	0610gsp9	structured analysis (a living risk graph)	104 files · 2.4 MB · 4 commits · 17 app entries	open live ↗
Risk-Zone for Health	0610gsp9	structured analysis (a living risk graph)	33 files · 428 KB · 7 commits · 1 app entry	open live ↗

keys blurred for the video, not on the site

Decrypted in your browser

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

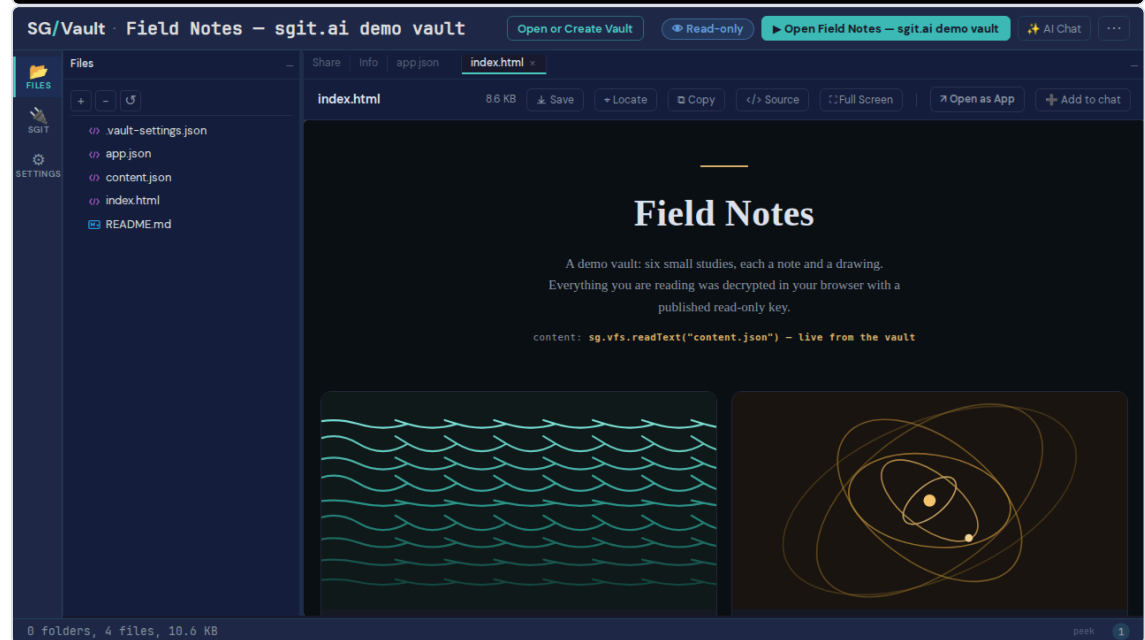
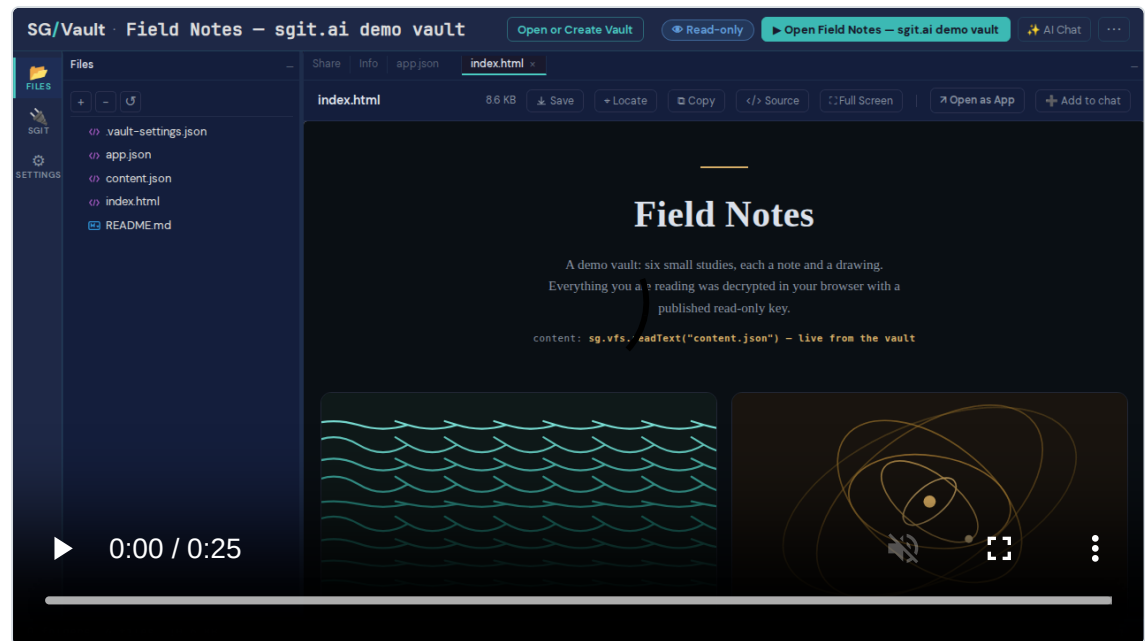
still · published vault “Field Notes” · spot rect ·
label “decrypted in the browser, with a published read
key”



Files, history, branches

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

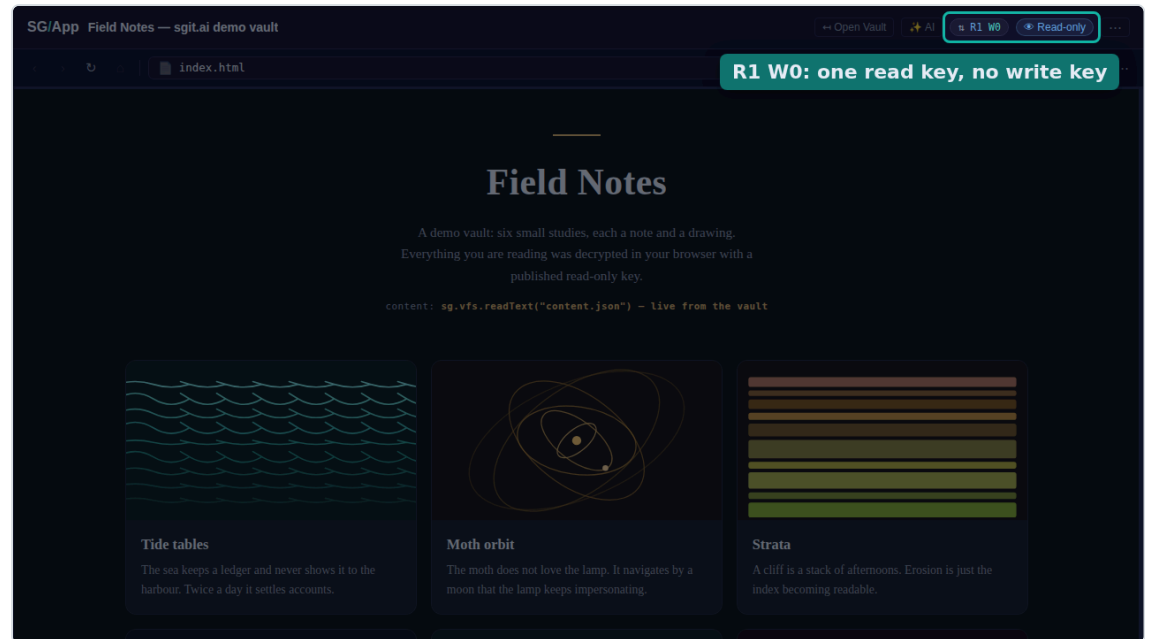
clip · published vault "Field Notes"



Read-only by construction

And the badge says what the key allows. One read, zero writes.

still · published vault “Field Notes” · spot badges ·
label “R1 W0: one read key, no write key”

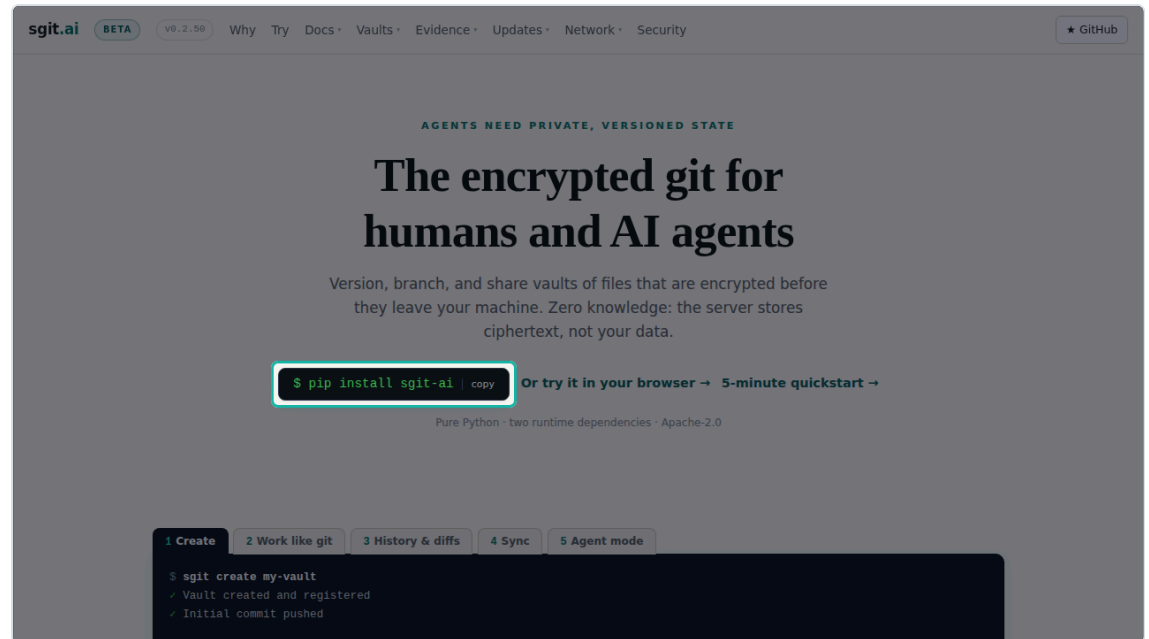


01:46 SCENE 13 OF 13 S13 7.3 S

pip install sgit-ai

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

still · <https://sgit.ai/> · spot pip



01:54 CLOSING 11.6 S

How this video was made

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

MADE BY a Claude Code agent, unattended

WHEN 2026-09-01

SCRIPT 13 scenes + title and closing · 269 words · reel.json written first

SCREENSHOTS 15 shot, 13 used · Playwright, headless · 93 s

NARRATION Kokoro-82M in the browser (2 workers), voice am_michael · 125.7 s of speech in 166 s · model load 7.6 s

RENDER video-creator, canvas + MediaRecorder, real time · 126 s

PIPELINE WALL CLOCK 394 s: capture 93 +
compose and narrate 176 + render 126

API COST \$0.00 · no LLM, TTS or image API
calls

COMPUTE one 4-core container · agent
session tokens not metered here

Transcript

The narration in order, as plain text, ready to paste next to the video.

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

Creating a vault is one command.

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

A vault can contain an app. Sharing the vault link is deploying the app.

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

And the badge says what the key allows. One read, zero writes.

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

Renders

CUT	LENGTH	SIZE	SLIDES	TTS	RECORD	DRIFT	PIPELINE
landscape 1280×720 • Kokoro	02:05.75	9.4 MB	15	166 s	125.9 s	176 ms	302 s
shorts 1080×1920 • Kokoro	01:09.94	6.5 MB	8	96 s	70.0 s	127 ms	177 s
landscape 1280×720 • OpenRouter gpt-audio	01:50.55	8.0 MB	15	5 s	110.6 s	160 ms	127 s

Source of truth is reel.json. Stills in images/ (desktop) and images-shorts/ (phone); clips in clips/. Timecodes are the landscape cut's TTS durations.